DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA16 Data Warehousing and Data Mining

Assignment 3 – CO3: Classification and Prediction

Name: Gondel Prashanth

Register No.: 192311366

Date of submission: 06/11/2026

NOTE

How this document is organised: every question is shown first (shaded blue box), followed by its answer. Each answer is split into Shared Group Work, Student A and Student B parts, as in the assignment.

About the experiments: Weka was not available in the environment used to prepare this document, and the credit-g file could not be downloaded. All hand calculations are exact. Where the assignment asks for Weka runs, the Weka procedure is given and the same experiments were run in **Python (scikit-learn)** on equivalent data (a 1,000-row credit-like synthetic set, the breast-cancer set, synthetic circles, synthetic imbalanced sets). Those numbers are real outputs of that code. Student B should repeat the Weka/credit-g runs and paste the screenshots next to the matching tables (blank 'Weka result' tables are provided).

Contents: Q1 Decision Trees · Q2 Naïve Bayes · Q3 Rules and Associative Classification · Q4 Backpropagation · Q5 Support Vector Machines · Q6 Evaluation · Q7 Design Challenge · Contribution Record

Q1 – Decision Tree Induction: Choosing the Splitting Attribute (6 Marks)

QUESTION

A bank records 12 loan decisions (Income, Credit History, Employment → Approved). Shared: identify class attribute, candidate attributes, stopping conditions; agree the splitting criterion and pruning; install Weka and load the data.

Student A (manual): entropy of the data set and information gain of every attribute at the root; gain ratio or Gini for one attribute (does it change the root?); complete tree as a set of paths; what happens with an attribute with many distinct values (customer ID).

Student B (Weka): run J48 on the 12 tuples and on credit-g; compare with Student A's tree; vary pruning confidence factor and minimum instances per leaf (report tree size and accuracy); explain overfitting with own evidence.

ANSWER

Shared Group Work

- **Class attribute:** Approved (Yes / No) – 5 Yes, 7 No.
- **Candidate splitting attributes:** Income {High, Medium, Low}, Credit History {Good, Poor}, Employment {Salaried, Self-employed}.
- **Stopping conditions:** (i) all tuples in the node belong to one class → leaf; (ii) no attributes left → leaf labelled with majority class; (iii) a branch has no tuples → leaf labelled with the majority class of the parent.
- **Splitting criterion:** information gain (ID3/C4.5); gain ratio and Gini index are used as a cross-check. **Pruning:** post-pruning (C4.5 pessimistic-error pruning, controlled by confidence factor C) plus a minimum number of instances per leaf (M).
- **Weka set-up:** Weka Explorer → Preprocess → Open file (loan.csv / loan.arff) → Classify → Choose trees ▸ J48 → Test options: Use training set for the 12 tuples, 10-fold cross-validation for credit-g.

Student A – Manual Induction

1. Entropy of the full data set

Info(D) = $-(5/12)\log_2(5/12) - (7/12)\log_2(7/12) = 0.5263 + 0.4536 =$ **0.980 bits**

2. Information gain of each attribute at the root

Attribute	Partition (Yes, No)	Entropy of each partition	Info_A(D)	Gain = 0.980 − Info_A
Income	High (3,1); Medium (1,3); Low (1,3)	0.811 ; 0.811 ; 0.811	(4/12)(0.811)×3 = 0.811	**0.169**
Credit History	Good (4,2); Poor (1,5)	0.918 ; 0.650	(6/12)(0.918)+(6/12)(0.650) = 0.784	**0.196**
Employment	Salaried (3,3); Self-emp. (2,4)	1.000 ; 0.918	(6/12)(1.000)+(6/12)(0.918) = 0.959	**0.021**

Example working: Credit = Good has 6 tuples (4 Yes, 2 No) → $-(4/6)\log_2(4/6) - (2/6)\log_2(2/6) = 0.390 + 0.528 = 0.918$. Credit = Poor has 6 tuples (1 Yes, 5 No) → 0.431 + 0.219 = 0.650.

Root attribute = Credit History (highest gain, 0.196).

3. Gain ratio and Gini index – does the root change?

Attribute	SplitInfo	Gain ratio = Gain/SplitInfo	Weighted Gini	Gini reduction (Gini(D)=0.486)
Income	$\log_2 3 = 1.585$	0.169/1.585 = 0.106	0.375	0.111
Credit History	1.000	0.196/1.000 = **0.196**	0.361	**0.125**
Employment	1.000	0.021	0.472	0.014

Gini(D) = 1 − (5/12)² − (7/12)² = 0.486. Both criteria rank Credit History first, so **the choice of root does not change**. Gain ratio does penalise Income (3 branches) relative to its raw gain, which is its purpose.

4. Complete tree

Level 2, Credit = Good (Info = 0.918; tuples 1,3,4,6,8,11): Gain(Employment) = 0.918 − 0.459 = **0.459**; Gain(Income) = 0.918 − 0.667 = 0.252 → split on Employment. Salaried → {1,4,8} all Yes (pure). Self-employed → {3 Yes, 6 No, 11 No}, split on Income: High → Yes, Medium → No, Low → No (all pure).

Level 2, Credit = Poor (Info = 0.650; tuples 2,5,7,9,10,12): Gain(Income) = 0.650 − 0.333 = **0.317**; Gain(Employment) = 0.650 − 0.459 = 0.191 → split on Income. Medium → No, Low → No (pure). High → {2 No (Salaried), 12 Yes (Self-employed)}, split on Employment.

```
Credit History?
├─ Good ── Employment?
│     ├─ Salaried ..................... YES   (tuples 1,4,8)
│     └─ Self-employed ── Income?
│          ├─ High ..................... YES   (tuple 3)
│          ├─ Medium ................... NO    (tuple 6)
│          └─ Low ...................... NO    (tuple 11)
└─ Poor ── Income?
     ├─ High ── Employment?
     │     ├─ Salaried ................ NO    (tuple 2)
     │     └─ Self-employed ........... YES   (tuple 12)
     ├─ Medium ....................... NO    (tuples 5,7)
     └─ Low .......................... NO    (tuples 9,10)
```

Tree as a set of paths (8 leaves, 12/12 training tuples correct):

- P1: Credit = Good ∧ Employment = Salaried → Approved = Yes
- P2: Credit = Good ∧ Employment = Self-employed ∧ Income = High → Yes
- P3: Credit = Good ∧ Employment = Self-employed ∧ Income = Medium → No
- P4: Credit = Good ∧ Employment = Self-employed ∧ Income = Low → No
- P5: Credit = Poor ∧ Income = High ∧ Employment = Salaried → No
- P6: Credit = Poor ∧ Income = High ∧ Employment = Self-employed → Yes
- P7: Credit = Poor ∧ Income = Medium → No
- P8: Credit = Poor ∧ Income = Low → No

5. Attribute with many distinct values (customer ID)

A unique ID splits the 12 tuples into 12 single-tuple partitions, each pure, so Info_ID(D) = 0 and Gain(ID) = 0.980 – the **maximum possible**, far above Credit History (0.196). Information gain is therefore biased toward many-valued attributes. The resulting tree would have the ID at the root with 12 leaves: it memorises the training set (100% training accuracy) but is useless for a new customer, whose ID has no branch – a textbook case of overfitting.

C4.5's gain ratio divides by SplitInfo(ID) = $\log_2 12 = 3.585$, giving 0.980/3.585 = 0.273. This reduces, but on this tiny data set does not remove, the bias (0.273 is still above Credit History's 0.196, and C4.5's 'average-gain' filter would still admit the ID). The practical remedy is to **remove identifier attributes during preprocessing**, plus pruning and minimum-leaf-size settings.

Student B – Weka Validation

Procedure and settings

- Classify → trees ▸ J48. Defaults: confidenceFactor C = 0.25, minNumObj M = 2, unpruned = False, binarySplits = False.
- Run 1: loan data, Test options = Use training set; right-click result → Visualize tree. Run 2: credit-g (1,000 instances, 20 attributes), 10-fold CV. Run 3–8: vary C ∈ {0.05, 0.1, 0.25, 0.5} and M ∈ {1, 2, 5, 10, 20, 50}; also run unpruned = True.

Weka result table for credit-g (Student B to complete from own run):

Setting	C	M	Tree size	Leaves	10-fold CV accuracy
Unpruned	–	2			
Default	0.25	2			
More pruning	0.05	2			
Less pruning	0.5	2			
Larger leaves	0.25	10			
Larger leaves	0.25	50			

Comparison of the manual tree with the Weka/software tree on the 12 tuples

The same data was fitted in Python with an entropy criterion. The root is again **Credit History**, confirming the manual calculation. The software tree (6 leaves, 100% training accuracy) is:

```
|--- Credit=Poor <= 0.50
|   |--- Emp=Salaried <= 0.50
|   |   |--- Income=High <= 0.50
|   |   |   |--- class: No
|   |   |--- Income=High >  0.50
|   |   |   |--- class: Yes
|   |--- Emp=Salaried >  0.50
|   |   |--- class: Yes
|--- Credit=Poor >  0.50
|   |--- Income=High <= 0.50
|   |   |--- class: No
|   |--- Income=High >  0.50
|   |   |--- Emp=Salaried <= 0.50
|   |   |   |--- class: Yes
|   |   |--- Emp=Salaried >  0.50
|   |   |   |--- class: No
```

Differences and why:

- **Binary vs multiway splits:** scikit-learn (and J48 with binarySplits = True) tests one value at a time (Income = High?), so the three-valued Income becomes one binary test, and the tree has 6 leaves instead of 8. Default J48 makes multiway splits on nominal attributes, like the manual tree.
- **Minimum leaf size:** J48's M = 2 requires at least two instances in at least two branches, so single-tuple leaves such as Good ∧ Self-employed ∧ High → Yes (tuple 3 only) cannot

be created; J48 will therefore usually give a smaller tree than the manual one. Expect the Weka tree to be pruned back after the first split(s) – record the actual size.

- **Criterion:** manual tree uses gain; J48 uses gain ratio. As shown above both choose Credit History at the root, so differences appear only at lower levels.
- **Pruning:** the manual tree is unpruned (100% training accuracy); J48's pessimistic-error pruning (C = 0.25) replaces subtrees whose estimated error is not better than a leaf. With 12 tuples every leaf is tiny, so pruning is aggressive.

Effect of pruning parameters (Python equivalent on a 1,000-row credit-like data set, 70/30 split)

M corresponds to min_samples_leaf; a stronger pruning setting (lower Weka C) corresponds to a larger cost-complexity penalty (ccp_alpha).

M (min leaf)	Leaves	Train acc.	Test acc.	Leaves (+pruning α=0.005)	Train acc.	Test acc.
1	71	100.0%	74.3%	68	99.6%	74.0%
2	70	97.9%	75.7%	57	97.4%	75.0%
5	51	94.4%	76.7%	45	92.7%	78.3%
10	35	88.9%	73.0%	31	88.9%	73.0%
20	25	88.0%	75.0%	21	88.0%	75.0%
50	11	84.4%	76.7%	11	84.4%	76.7%

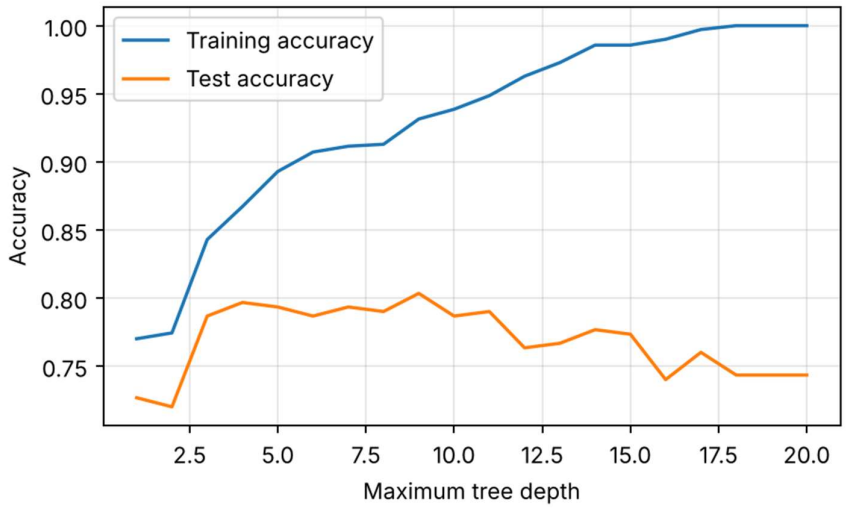


Figure 1.1 – Training accuracy keeps rising with tree depth while test accuracy stays flat or falls.

Overfitting explained with the evidence: the fully grown tree (M = 1) has 71 leaves and 100% training accuracy but only 74.3% test accuracy – a 25.7-point gap. Increasing M to 5 reduces the tree to 51 leaves (45 with pruning) and raises test accuracy to 76.7–78.3% even though training accuracy drops to 94%. Beyond M ≈ 20 the tree becomes too small (11 leaves, 84% training) and starts under-fitting. The model that fits training data best is therefore not the one that generalises best; deep branches are fitting noise.

Individual Verification

Each student should be ready to explain: why Credit History is the root (gain 0.196), why the many-valued ID attribute gets maximum gain, and what changes in J48 when M is raised from 2 to 20.

Q2 – Bayesian Classification and the Cost of Assumptions (6 Marks)

QUESTION

Using the loan data of Q1, classify X = (Income = Low, Credit History = Good, Employment = Self-employed) with Naïve Bayes.

Student A: priors and conditional probabilities; posterior (proportional) value for each class and predicted class; show what happens when a conditional probability is zero and fix with Laplacian correction.

Student B: run NaiveBayes in Weka and verify; create a data set with two strongly correlated attributes and measure how Naïve Bayes accuracy changes vs a decision tree; explain why Naïve Bayes still gives useful class rankings.

Joint: when would you prefer Naïve Bayes over a decision tree, and when not?

ANSWER

Student A – Calculation

Prior probabilities

P(Yes) = 5/12 = 0.4167 P(No) = 7/12 = 0.5833

Conditional probabilities

Attribute value	P(value | Yes) (5 tuples)	P(value | No) (7 tuples)
Income = Low	1/5 = 0.200 (tuple 8)	3/7 = 0.429 (tuples 9,10,11)
Credit History = Good	4/5 = 0.800 (tuples 1,3,4,8)	2/7 = 0.286 (tuples 6,11)
Employment = Self-employed	2/5 = 0.400 (tuples 3,12)	4/7 = 0.571 (tuples 5,6,9,11)

Posterior (proportional) values

P(X|Yes)·P(Yes) = 0.2 × 0.8 × 0.4 × 0.4167 = **0.0267**

P(X|No)·P(No) = 0.429 × 0.286 × 0.571 × 0.5833 = (24/343) × 0.5833 = **0.0408**

Normalised: P(Yes|X) = 0.0267/(0.0267+0.0408) = 0.395, P(No|X) = 0.605. **Predicted class: Approved = No.**

Zero conditional probability and Laplacian correction

For the full data no needed probability is zero, so the problem is shown with a hypothetical: suppose tuple 8 (the only Low-income Yes tuple) were missing from the training set. Then Yes has 4 tuples and P(Income = Low | Yes) = 0/4 = 0.

- Without correction: P(X|Yes) = 0 × 0.75 × 0.5 = 0, so the Yes posterior is **0** whatever the other attributes say; one unseen combination vetoes the class (prediction No, 0.0445 vs 0).
- **Laplacian correction** (add 1 to every count; denominator + number of distinct values): P(Low|Yes) = (0+1)/(4+3) = 1/7; P(Good|Yes) = (3+1)/(4+2) = 4/6; P(Self|Yes) = (2+1)/(4+2) = 3/6. For No (7 tuples): P(Low|No) = 4/10, P(Good|No) = 3/9, P(Self|No) = 5/9. Priors 4/11 and 7/11.
- Recomputed: Yes = (4/11)(1/7)(4/6)(3/6) = **0.0173**; No = (7/11)(4/10)(3/9)(5/9) = **0.0471** → P(Yes|X) = 0.27, P(No|X) = 0.73. Yes is no longer eliminated; the prediction is still No.

Applying the same correction to the original 12 tuples: Yes = 0.0319, No = 0.0432 → P(Yes|X) = 0.425, P(No|X) = 0.575 (still No).

Student B – Investigation

Weka verification

Classify → bayes ▸ NaiveBayes → Use training set. Weka's NaiveBayes initialises every count to 1 (a built-in Laplace estimator) including the class priors, so its printed values should match the **Laplace-corrected** column below, not the raw counts:

Quantity	Manual (raw counts)	Weka-style (Laplace, +1)
P(Yes) / P(No)	0.417 / 0.583	6/14 = 0.429 / 8/14 = 0.571
P(Low | Yes) ; P(Low | No)	0.200 ; 0.429	2/8 = 0.250 ; 4/10 = 0.400
P(Good | Yes) ; P(Good | No)	0.800 ; 0.286	5/7 = 0.714 ; 3/9 = 0.333
P(Self | Yes) ; P(Self | No)	0.400 ; 0.571	3/7 = 0.429 ; 5/9 = 0.556
Posterior for X (Yes ; No)	0.395 ; 0.605	0.437 ; 0.563

Both agree on the predicted class (No); the small numerical differences are due only to the smoothing. (Student B: attach the Weka output screenshot and check the figures.)

Correlated-attribute experiment

Design: a balanced two-class synthetic set (2,000 tuples). One strong independent attribute b, plus a weaker informative attribute a that is present as 1, 2, 3, 5 or 9 near-identical copies (correlation ≈ 0.99 between copies). Gaussian Naïve Bayes and a decision tree (min leaf 20) were compared by 5-fold CV (averaged over 5 data sets).

Copies of correlated attribute	Naïve Bayes accuracy	Decision tree accuracy	Naïve Bayes AUC
1 (no correlation)	74.1%	71.8%	0.821
2	72.8%	71.9%	0.808
3	71.2%	71.4%	0.788
5	68.9%	70.9%	0.755
9	66.2%	70.7%	0.721

Result: with independent attributes Naïve Bayes is better than the tree (74.1% vs 71.8%). As the same evidence is repeated, Naïve Bayes falls steadily to 66.2% and its AUC from 0.82 to 0.72, while the decision tree stays almost unchanged (70.7%) because it picks one copy and ignores the rest. Naïve Bayes multiplies the duplicated likelihoods, so the correlated attribute ends up counted many times and out-votes the stronger independent one.

Why Naïve Bayes can still give useful rankings

- Classification only needs the **order** of P(Yes|X) and P(No|X), not their exact values. Duplicated evidence mostly pushes both posteriors toward extremes (over-confidence) in the same direction, so many tuples are still ranked correctly – the independence violation distorts probabilities more than ranks.
- If dependencies among attributes are similar in both classes they partly cancel in the likelihood ratio.
- Naïve Bayes has high bias but very low variance (few parameters), which pays off on small data; the AUC stays well above 0.5 in the experiment even at 9 copies.

Joint Conclusion

Prefer Naïve Bayes when…	Prefer a decision tree when…
Training data is small or has many attributes (text, spam, medical screening)	Attributes are strongly correlated or interact (e.g., class depends on A AND B)

Prefer Naïve Bayes when…	Prefer a decision tree when…
Attributes are roughly conditionally independent	Human-readable rules/path explanations are required
Probabilistic scores/rankings, fast training and incremental updating are needed	Mixed data, missing values and non-linear thresholds should be handled directly
Missing values can simply be ignored in the product	Enough data exists to learn interactions without overfitting

Q3 – Rule-Based and Associative Classification (7 Marks)

QUESTION

Shared: convert the Q1 tree into IF–THEN rules; discuss how rules represent knowledge differently from a tree.

Student A: extract rules, compute coverage and accuracy of each on the 12 tuples; explain conflict resolution (size ordering, rule ordering) with an example; explain sequential covering and what is removed after each rule.

Student B: run JRip (or PART) on credit-g and compare with J48; explain associative classification, support and confidence; mine class-association rules from the 12 tuples (min support 25%, confidence 70%) and list those predicting Approved = Yes.

Joint: a loan officer must explain every rejection – which of tree, rules, associative rules do you recommend?

ANSWER

Shared Group Work – tree to rules

One rule is created per root-to-leaf path (AND of the tests on the path, leaf = consequent). Rules from a tree are **mutually exclusive** (each tuple satisfies exactly one) and **exhaustive**. Unlike a tree, a rule set can be read, edited or deleted rule by rule, has no fixed order of tests, can drop conditions that matter only along one path, and can be made more compact (rules need not share the same attribute order); a tree gives a single global hierarchy where the first test is shared by every case.

Student A – Rule Quality

coverage(R) = n_covers/|D|; accuracy(R) = n_correct/n_covers (|D| = 12).

Rule	IF … THEN …	Covers (tuples)	Coverage	Accuracy
R1	Credit = Good ∧ Employment = Salaried → Yes	3 (1,4,8)	25.0%	3/3 = 100%
R2	Credit = Good ∧ Employment = Self ∧ Income = High → Yes	1 (3)	8.3%	1/1 = 100%
R3	Credit = Good ∧ Employment = Self ∧ Income = Medium → No	1 (6)	8.3%	100%
R4	Credit = Good ∧ Employment = Self ∧ Income = Low → No	1 (11)	8.3%	100%
R5	Credit = Poor ∧ Income = High ∧ Employment = Salaried → No	1 (2)	8.3%	100%
R6	Credit = Poor ∧ Income = High ∧ Employment = Self → Yes	1 (12)	8.3%	100%
R7	Credit = Poor ∧ Income = Medium → No	2 (5,7)	16.7%	2/2 = 100%
R8	Credit = Poor ∧ Income = Low → No	2 (9,10)	16.7%	2/2 = 100%

Total coverage = 12/12; all rules are 100% accurate **on the training data** (which says nothing about test accuracy – several rules rest on a single tuple).

Rule conflict resolution – example from the data

Conflicts arise when a rule set is **not** mutually exclusive (e.g., rules learned directly, or a tree's rules after pruning). Consider two rules:

- Ra: IF Credit = Poor THEN Approved = No (covers 2,5,7,9,10,12; accuracy 5/6 = 83%)
- Rb: IF Income = High ∧ Employment = Self-employed THEN Approved = Yes (covers 3,12; accuracy 2/2 = 100%)

Tuple 12 (High, Poor, Self-employed) fires both, with opposite classes.

- **Size ordering:** give priority to the rule with the larger number of attribute tests (the 'toughest' requirement). Rb has 2 tests, Ra has 1 → Rb fires → **Yes** (correct).
- **Rule ordering:** rules are ranked in advance (by accuracy, coverage or size) as a decision list, and the first rule that fires decides. If Ra is ranked first (it has larger coverage) the prediction is **No** (wrong); if Rb is ranked first (higher accuracy) it is Yes. Class-based ordering ranks all rules of a class together by prevalence/misclassification cost. A **default rule** (ELSE class) handles tuples no rule covers.

Sequential covering

Rules are learned **one class and one rule at a time** (AQ, CN2, FOIL, RIPPER): (1) Learn-One-Rule starts from the empty rule IF true THEN Yes and greedily adds the attribute test that most improves accuracy/FOIL gain, until the rule is accurate enough; (2) the rule is accepted; (3) **the tuples covered by the rule are removed** from the training set (positive tuples in particular), so the next rule concentrates on what is still unexplained; (4) repeat until no positives remain or rule quality falls below a threshold. In contrast to tree induction (divide-and-conquer on all classes at once), rules are learned sequentially.

On the loan data (target Yes):

- Rule 1: true → Yes (5/12); add Credit = Good → 4/6; add Employment = Salaried → 3/3. **Rule 1: IF Good ∧ Salaried THEN Yes.** Remove tuples 1,4,8.
- Remaining Yes: 3, 12. Rule 2: add Income = High → 2/3 (tuples 2,3,12); add Employment = Self-employed → 2/2. **Rule 2: IF High ∧ Self-employed THEN Yes.** Remove tuples 3,12.
- No Yes tuples remain → default rule **ELSE No** (7/7 correct). The compact list of 2 rules + default classifies all 12 tuples correctly, versus 8 rules from the tree.

Student B – Associative Classification

JRip / PART on credit-g in Weka

- Classify → rules ▸ JRip (folds = 3, minNo = 2.0, optimizations = 2) and rules ▸ PART; Test options: 10-fold cross-validation; run J48 with the same options.
- Record in a table: number of rules, accuracy, and 2–3 example rules from each. Compare: JRip produces a short **ordered decision list** (typically a handful of rules for the minority 'bad' class with a default 'good'), PART produces rules derived from partial pruned trees; J48 yields a larger tree (≈ 100 nodes unpruned-to-default) whose paths are mutually exclusive. Rules from JRip are normally far fewer and more readable, at similar (≈ 70–75%) accuracy. (Student B: confirm these figures from your own run.)

Classifier	No. of rules / leaves	10-fold accuracy	Example rule
J48			
JRip			
PART			

How associative classification differs

Associative classification (CBA, CMAR, CPAR) first **mines all class-association rules (CARs)** of the form (condset) → class using Apriori-style frequent-pattern mining, keeps those that pass

minimum support and confidence, ranks them (confidence, then support, then shorter rule) and builds a classifier from the best rules (CBA uses the first matching rule; CMAR combines several). Rules from trees or sequential covering are built greedily, so they see only one path of the search space; associative classification searches **exhaustively** for all strong rules, so overlapping rules and combinations of attributes a greedy method would miss are found, at the cost of many rules and more computation.

- **Support** = % of all tuples that contain both the condition set and the class: sup = count(condset ∧ class)/|D|. It measures how general the rule is and filters out rules based on a few tuples.
- **Confidence** = count(condset ∧ class)/count(condset) – the accuracy of the rule on the tuples it covers. It filters out unreliable rules and is also used to rank rules.

Class-association rules from the 12 tuples (min support 25% = 3 tuples, min confidence 70%)

Counting only tuples with Approved = Yes (1,3,4,8,12):

Rule item	Count (cond ∧ Yes)	Support	Cond. count	Confidence	Result
Income = High → Yes	3	25.0%	4	75.0%	**Keep**
Income = Medium → Yes	1	8.3%	4	25%	Fails support
Income = Low → Yes	1	8.3%	4	25%	Fails support
Credit = Good → Yes	4	33.3%	6	66.7%	Fails confidence
Credit = Poor → Yes	1	8.3%	6	16.7%	Fails support
Employment = Salaried → Yes	3	25.0%	6	50.0%	Fails confidence
Employment = Self-emp. → Yes	2	16.7%	6	33.3%	Fails support
Credit = Good ∧ Employment = Salaried → Yes	3	25.0%	3	100%	**Keep**
Income = High ∧ Credit = Good → Yes	2	16.7%	2	100%	Fails support
Income = High ∧ Employment = Self → Yes	2	16.7%	2	100%	Fails support

Class-association rules predicting Approved = Yes:

- **CAR1:** IF Income = High THEN Approved = Yes [support 25%, confidence 75%]
- **CAR2:** IF Credit History = Good ∧ Employment = Salaried THEN Approved = Yes [support 25%, confidence 100%]

No three-condition rule reaches 3 tuples. Note that CAR1 covers tuple 2 (High, Poor, Salaried → No) wrongly – a rule can be ‘strong’ without being perfect.

Joint Design Question – recommendation

Recommended: IF–THEN rules derived from a pruned decision tree (a short decision list).
Reasons:

- Every rejection is triggered by exactly **one** rule, so the officer can state the reason: ‘Your credit history is poor and your income is not high.’ Tree rules are mutually exclusive and need no conflict resolution.
- A tree in its graphical form is equally explainable but long paths are harder to read than separate rules and cannot be shortened.
- Associative rules are strong for describing approvals, but several rules can fire with different classes, so the officer would need to justify the conflict-resolution rule; thousands of rules also hinder audit. They can be used as a supplement to find additional patterns.
- For fairness and regulation, the rule base should be kept short, pruned and tested on unseen data.

Q4 – Backpropagation: What Does the Network Learn? (7 Marks)

QUESTION

Network: 2 inputs, 2 hidden units, 1 output, sigmoid. x1 = 1, x2 = 0, target t = 1, learning rate 0.9. Weights: w(x1→h1) = 0.2, w(x2→h1) = 0.4, w(x1→h2) = −0.3, w(x2→h2) = 0.1, w(h1→o) = −0.3, w(h2→o) = −0.2; biases h1 = −0.4, h2 = 0.2, o = 0.1.

Student A: forward pass; error terms; update all weights/biases for one iteration and recompute the output (did error decrease?); why weights start small and random and what happens if all are equal.

Student B: train a MultilayerPerceptron (Weka or Python) on a real data set; vary learning rate, hidden units, epochs; show overfitting and slow/unstable training; compare training time and interpretability with the Q1 tree.

ANSWER

Student A – One Training Step

Forward pass (O = 1/(1+e^(−I)))

Unit	Net input I	Output O
h1	0.2(1) + 0.4(0) − 0.4 = −0.200	1/(1+e^0.2) = **0.4502**
h2	−0.3(1) + 0.1(0) + 0.2 = −0.100	1/(1+e^0.1) = **0.4750**
o	−0.3(0.4502) − 0.2(0.4750) + 0.1 = −0.1301	1/(1+e^0.1301) = **0.4675**

Error before the step: t − O = 1 − 0.4675 = 0.5325 (squared error ½(t−O)² = 0.1418).

Error terms (backward pass)

- Output: Err_o = O(1−O)(t−O) = 0.4675 × 0.5325 × 0.5325 = **0.1326**
- h1: Err_h1 = O_h1(1−O_h1) × Err_o × w(h1→o) = 0.4502 × 0.5498 × 0.1326 × (−0.3) = **−0.00984**
- h2: Err_h2 = O_h2(1−O_h2) × Err_o × w(h2→o) = 0.4750 × 0.5250 × 0.1326 × (−0.2) = **−0.00661**

Updates (Δw = l · Err_j · O_i , Δθ = l · Err_j , l = 0.9)

Weight / bias	Old	Change	New
w(h1→o)	−0.3000	0.9 × 0.1326 × 0.4502 = +0.0537	**−0.2463**
w(h2→o)	−0.2000	0.9 × 0.1326 × 0.4750 = +0.0567	**−0.1433**
w(x1→h1)	0.2000	0.9 × (−0.00984) × 1 = −0.0089	**0.1911**
w(x2→h1)	0.4000	0.9 × (−0.00984) × 0 = 0	**0.4000**
w(x1→h2)	−0.3000	0.9 × (−0.00661) × 1 = −0.0059	**−0.3060**
w(x2→h2)	0.1000	0 (x2 = 0)	**0.1000**
bias o	0.1000	0.9 × 0.1326 = +0.1193	**0.2193**
bias h1	−0.4000	0.9 × (−0.00984) = −0.0089	**−0.4089**
bias h2	0.2000	0.9 × (−0.00661) = −0.0059	**0.1940**

Recomputed output

h1: I = 0.1911 − 0.4089 = −0.2177 → O = 0.4458; h2: I = −0.3060 + 0.1940 = −0.1119 → O = 0.4721; o: I = −0.2463(0.4458) − 0.1433(0.4721) + 0.2193 = 0.0418 → **O = 0.5105**.

The error fell from 0.5325 to 0.4895 (squared error 0.1418 → 0.1198). **Yes, the error decreased**, and the output moved from 0.4675 toward the target 1.

Why small random initial weights?

- **Symmetry breaking:** if all weights are identical, h1 and h2 compute identical outputs and receive identical error terms and updates, so they stay identical forever – the two hidden units act as one and the network cannot learn different features. Random values make the units different.
- **Small values** keep the net inputs near 0, where the sigmoid is steep (O(1−O) ≈ 0.25), so gradients are large and learning starts quickly. Large weights saturate the sigmoid (O near 0 or 1, derivative ≈ 0) → vanishing gradients and very slow learning.

Student B – Behaviour of the Learner

Data set: Breast Cancer Wisconsin (569 tuples, 30 numeric attributes, 2 classes), 70/30 stratified split, attributes standardised. Network: SGD back-propagation with momentum 0.9 (Python MLPClassifier, 1 hidden layer). Default 5 hidden units, learning rate 0.1, 200 epochs unless varied. (Weka equivalent: classifiers ▸ functions ▸ MultilayerPerceptron with learningRate, hiddenLayers and trainingTime; Weka defaults are l = 0.3, momentum = 0.2, 500 epochs.)

Effect of learning rate

Learning rate	Train acc.	Test acc.	Final training loss
0.0001	83.9%	81.9%	0.4496
0.001	97.2%	91.2%	0.1415
0.01	99.2%	95.3%	0.0377
0.1	99.7%	94.2%	0.0068
0.5	99.7%	94.7%	0.0030
1	100.0%	94.7%	0.0003

Effect of number of hidden units

Hidden units	Train acc.	Test acc.	Training time (s)
1	99.2%	95.9%	0.05
2	99.5%	94.2%	0.05
5	99.7%	94.2%	0.05
10	100.0%	95.3%	0.06
50	100.0%	95.3%	0.09
200	100.0%	95.3%	0.32

Effect of number of epochs

Epochs	Train acc.	Test acc.	Final training loss
2	95.0%	90.6%	0.3508
5	97.2%	91.2%	0.1206

Epochs	Train acc.	Test acc.	Final training loss
10	98.2%	95.3%	0.0664
50	99.5%	94.2%	0.0256
200	99.7%	94.2%	0.0068
1000	100.0%	96.5%	0.0015

Overfitting, slow and unstable training

- **Slow training:** at l = 0.0001 the network is still at 83.9% train / 81.9% test after 200 epochs (loss 0.450); at l = 0.001 it needs far more epochs to reach 97%. Likewise only 2 epochs gives 95.0% / 90.6%.
- **Unstable training:** with l = 3 the loss rises in the second epoch (0.44 → 0.55) before settling; with **l = 5 the loss diverges** (0.51, 4.08, 15.6, 703 … > 10^9) and with l ≥ 10 the weights overflow and training aborts. Too large a step overshoots the minimum.

Epochs (60 training tuples, 100 hidden units)	Train acc.	Test acc.	Gap
5	96.7%	91.8%	4.9 pts
20	98.3%	94.2%	4.2 pts
100	100.0%	94.2%	5.8 pts
500	100.0%	94.7%	5.3 pts
2000	100.0%	94.7%	5.3 pts

- **Overfitting:** with only 60 training tuples and a 100-unit hidden layer, training accuracy reaches 100% while test accuracy stays at ≈ 94–95%, a persistent gap of about 5 points; more epochs only memorise the training set (loss → 0). On this easy data set the test accuracy does not decrease, so the overfitting is mild; it is typically stronger on noisy data such as credit-g. Remedies: early stopping with a validation set, fewer hidden units, weight decay, more data.

Comparison with the decision tree of Q1

	Decision tree (unpruned)	MLP (5 hidden, 200 epochs)
Train / test accuracy	100.0% / 90.6%	99.7% / 94.2%
Training time	≈ 0.006 s	≈ 0.05 s (grows with hidden units/epochs: 0.32 s for 200 units)
Interpretability	Rules readable by a human	Black box: weights have no direct meaning
Needs scaling / tuning	No scaling; few parameters	Scaling needed; l, hidden units, epochs must be tuned

Strengths of neural classifiers: model non-linear relationships and interactions, tolerate noisy data, work with continuous inputs, good accuracy (here 94–96% vs 90.6% for the tree).
Limitations: long training, many parameters to tune, risk of local minima/overfitting, and poor explainability.

Individual Verification – how the error reaches a hidden-layer weight

The output error term $Err_o = O(1-O)(t-O)$ is multiplied by the weight $w(h \rightarrow o)$ that connects the hidden unit to the output, giving the hidden unit's share of the blame, then multiplied by the hidden unit's own sigmoid derivative: $Err_h = O_h(1-O_h) \cdot Err_o \cdot w_{ho}$. The weight from input

i to that hidden unit changes by $\Delta w_{ih} = l \cdot Err_h \cdot x_i$. In the example $Err_{h1} = -0.0098$ and $x1 = 1$, so $w(x1 \rightarrow h1)$ falls by 0.0089, while $w(x2 \rightarrow h1)$ is unchanged because $x2 = 0$.

Q5 – Support Vector Machines: Margin, Kernels and Scaling (7 Marks)

QUESTION

Six points in 2-D: Class +1 → (3,3), (4,3), (4,4); Class −1 → (1,1), (1,2), (2,1). Shared: plot and discuss the maximum-margin separator.

Student A: support vectors by inspection; a separating hyperplane w·x + b = 0 equidistant from the two classes; margin width; what changes if a non-support vector / a support vector is removed; role of soft-margin C and kernels.

Student B: SMO in Weka (or SVC in Python) on a non-linear data set with linear, polynomial and RBF kernels; effect of C and kernel parameters on accuracy and number of support vectors; effect of not scaling and then scaling.

Joint: for which type of data would you choose an SVM; practical limitations?

ANSWER

Shared Group Work – plot and maximum-margin idea

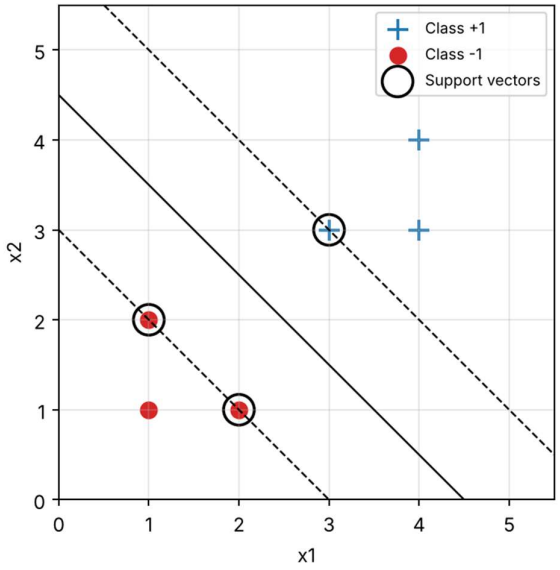


Figure 5.1 – Data, support vectors (circled), hyperplane x1 + x2 = 4.5 (solid) and margin boundaries (dashed).

Many lines separate the two classes. The **maximum-margin hyperplane** is the one with the largest distance to the nearest training point on either side: the larger the margin, the lower the capacity/variance, and (by structural risk minimisation) the better the expected generalisation. Only the closest points – the support vectors – determine it.

Student A – Geometry

- **Support vectors:** (3,3) from class +1, and (2,1) and (1,2) from class −1. (Distances from (3,3): √5 to both (2,1) and (1,2), but √8 to (1,1); (4,3) and (4,4) are farther away.)
- **Hyperplane:** set w·x + b = +1 at (3,3) and −1 at (2,1) and (1,2). The last two give $w_1 = w_2 = a$; then 6a + b = 1 and 3a + b = −1 → a = 2/3, b = −3. So **w = (2/3, 2/3), b = −3**, i.e. **x1 + x2 − 4.5 = 0** (any multiple, e.g. w = (1,1), b = −4.5, is the same line). Check: (3,3) → +1; (2,1), (1,2) → −1; (4,3) → +1.67, (4,4) → +2.33, (1,1) → −1.67, all correctly outside the margin.
- **Verification of optimality:** $w = \Sigma \alpha_i y_i x_i$ with α(3,3) = 4/9, α(2,1) = α(1,2) = 2/9, all positive and $\Sigma \alpha_i y_i = 0$ (4/9 − 2/9 − 2/9 = 0), satisfying the KKT conditions.

- **Margin width** = 2/‖w‖ = 2/(2√2/3) = 3/√2 = **2.12** (distance of each support vector from the line = 1.5/√2 = 1.06).
- **Removing a non-support vector** ((4,3), (4,4) or (1,1)): nothing changes – the hyperplane and margin are identical, since they depend only on the support vectors.
- **Removing a support vector:** the solution changes. E.g., removing (3,3) leaves (4,3) as the nearest positive point, so the hyperplane moves toward the positive class and the margin becomes wider; removing (2,1) or (1,2) rotates the hyperplane and changes the margin.
- **Soft-margin parameter C:** permits some points inside the margin or misclassified, with slack penalty $C \cdot \Sigma \xi_i$. Large C → narrow margin, few violations, risk of overfitting noise; small C → wide margin, more violations, smoother model (underfitting if too small).
- **Kernels:** when classes are not linearly separable, the kernel $K(x,z) = \varphi(x) \cdot \varphi(z)$ implicitly maps data to a higher-dimensional space where a linear separator exists, without computing φ. Common: polynomial $(x \cdot z + 1)^d$, RBF $\exp(-\gamma \|x-z\|^2)$; the separator is non-linear in the original space.

Student B – Experiments

Data: concentric circles (300 points, noise 0.15, factor 0.4), 70/30 split; SVC in Python (SMO-type solver; Weka: classifiers ▸ functions ▸ SMO with kernel PolyKernel / RBFKernel, c = complexity).

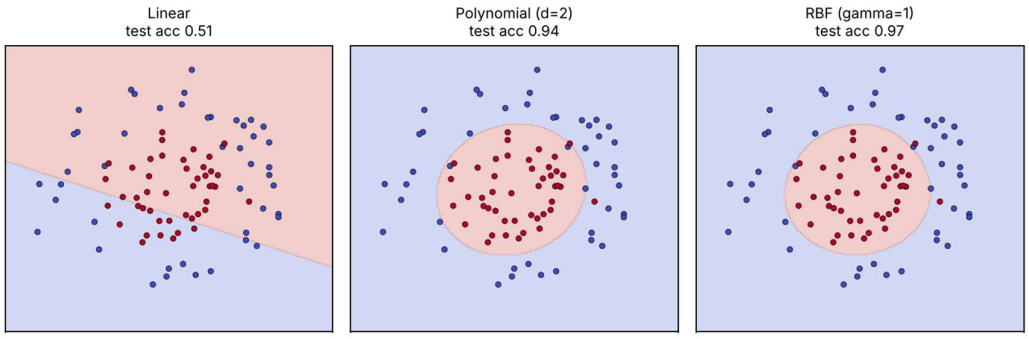


Figure 5.2 – Decision regions for linear, polynomial and RBF kernels (test points shown).

Kernel (C = 1)	Train acc.	Test acc.	Support vectors
Linear	59.0%	51.1%	208
Polynomial (d=2)	97.1%	94.4%	38
RBF (gamma=1)	97.1%	96.7%	57

The linear kernel is at chance level (51%) – no straight line separates concentric circles; polynomial and RBF reach 94–97%.

Effect of C (RBF, γ = 1)

C	Train acc.	Test acc.	Support vectors
0.01	96.2%	95.6%	210
0.1	96.2%	94.4%	148
1	97.1%	96.7%	57
10	97.6%	96.7%	26
100	98.6%	94.4%	19
1000	99.5%	95.6%	12

Larger C fits the training data harder (96.2% → 99.5%) and needs fewer support vectors (210 → 12) because fewer points lie inside the margin, but test accuracy peaks at C = 1–10 and then falls – the overfitting direction.

Effect of kernel parameter γ (RBF, C = 1)

γ	Train acc.	Test acc.	Support vectors
0.01	67.1%	57.8%	210
0.1	89.0%	88.9%	204
1	97.1%	96.7%	57
10	99.0%	94.4%	69
100	100.0%	93.3%	183
1000	100.0%	74.4%	206

Very small γ gives an almost linear, under-fitted model (test 57.8%); very large γ makes each point its own island (train 100%, test 74.4%, nearly all points become support vectors). γ ≈ 1 is best.

Effect of scaling the attributes

Data / kernel	Test acc. unscaled	Test acc. scaled (z-score)	Support vectors (unscaled → scaled)
Breast cancer, linear	94.2%	95.3%	35 → 27
Breast cancer, RBF	90.6%	95.3%	105 → 96
Toy: one useful attribute (≈ 0–1) + one noise attribute in thousands, RBF	51.1%	85.6%	–

Without scaling, distances (and the margin) are dominated by the attribute with the largest range; in the toy data the noise attribute measured in thousands swamps the useful one and accuracy falls to chance (51.1%), while scaling restores 85.6%. (Weka's SMO normalises by default; try filter = No normalization to reproduce.)

Joint Conclusion

- **Choose an SVM** for high-dimensional data with a moderate number of tuples (text, gene expression, images), where classes are separable by a margin/kernel and accuracy matters more than interpretability.
- **Limitations seen:** sensitive to scaling; accuracy depends strongly on C and γ (tuning by cross-validation needed); the number of support vectors, and therefore prediction time, grows when γ or C is poorly chosen; no direct explanation for decisions; training scales roughly quadratically with the number of tuples.

Q6 – Evaluating Classifiers and Predictors (7 Marks)

QUESTION

A fraud-detection classifier tested on 1,000 transactions: TP = 40, FN = 20, FP = 30, TN = 910. Shared: which measure is meaningful?

Student A: accuracy, error rate, sensitivity, specificity, precision, F-measure; show that "always not fraud" has high accuracy (class imbalance); MAE, MSE, RMSE for actual {12,15,20,25} vs predicted {10,18,19,30} and when RMSE is preferred; how a ROC curve is constructed and read.

Student B: compare holdout, 10-fold CV and bootstrap (.632) on an imbalanced set; repeat holdout ≥ 5 random splits and report variation; compare J48, Naïve Bayes and SMO with paired evaluation (Weka Experimenter); plot a ROC curve.

ANSWER

Shared Group Work

Only 60 of 1,000 transactions are fraud (6%). The meaningful measures are **recall/sensitivity** (how many frauds are caught), **precision** (how many alerts are real), the F-measure and the ROC/AUC – not accuracy, which is dominated by the 940 legitimate transactions.

Student A – Metrics and Prediction Error

Measure	Formula	Value
Accuracy	(TP+TN)/(P+N) = (40+910)/1000	**0.950 (95.0%)**
Error rate	(FP+FN)/1000 = 50/1000	**0.050 (5.0%)**
Sensitivity (recall)	TP/(TP+FN) = 40/60	**0.667 (66.7%)**
Specificity	TN/(TN+FP) = 910/940	**0.968 (96.8%)**
Precision	TP/(TP+FP) = 40/70	**0.571 (57.1%)**
F-measure (F1)	2PR/(P+R) = 2TP/(2TP+FP+FN) = 80/130	**0.615**

Class imbalance

A classifier that predicts 'not fraud' for every transaction has TP = 0, FN = 60, FP = 0, TN = 940: accuracy = 940/1000 = **94.0%**, almost as good as the real classifier (95.0%), but recall = 0, precision undefined, and it catches no fraud at all. When one class is rare, accuracy is dominated by the majority class, so it must be complemented by recall, precision, F-measure, ROC/AUC or a cost-based measure; remedies include resampling, class weighting and threshold adjustment.

Prediction error measures

Actual	Predicted	Error (y−ŷ)	|Error|	Error²
12	10	2	2	4
15	18	−3	3	9
20	19	1	1	1
25	30	−5	5	25
Total			**11**	**39**

- **MAE** = 11/4 = **2.75**
- **MSE** = 39/4 = **9.75**
- **RMSE** = √9.75 = **3.12** (same units as the target)

RMSE is preferred over MAE when large errors are particularly costly: squaring weights big errors more heavily (the error of 5 contributes 64% of the squared total but 45% of the absolute total), RMSE is differentiable and is the quantity minimised in least-squares fitting, and it is in the original units. MAE is more robust to outliers and easier to interpret as the average miss.

ROC curve

A classifier gives a score (e.g. P(fraud)). For each possible threshold, tuples scoring above it are called positive; compute the TPR = TP/(TP+FN) and FPR = FP/(FP+TN) and plot the points (FPR, TPR), joining them from (0,0) (threshold above every score) to (1,1) (threshold below every score). **Reading it:** the diagonal is random guessing; the closer the curve bends to the top-left corner, the better; the **area under the curve (AUC)** is the probability that a random positive receives a higher score than a random negative (0.5 = random, 1 = perfect). Moving along the curve shows the trade-off between catching more frauds (TPR) and more false alarms (FPR). The given classifier is a single point: FPR = 30/940 = 0.032, TPR = 0.667.

Student B – Estimation Methods

Data: synthetic imbalanced set, 1,000 tuples, 20 attributes, 12% minority class (majority-class baseline accuracy = 88.0%). Classifier: J48-like decision tree (entropy, min leaf 5). (Weka: credit-g gives 70/30 imbalance – repeat there and compare.)

Holdout with five random 70/30 splits

Split	1	2	3	4	5	Mean ± SD
Accuracy	85.0%	89.0%	88.3%	88.0%	85.3%	87.1% ± 1.8

Accuracy varies from 85.0% to 89.0% (a 4.0-point range) only because of how the data were split – a single holdout estimate is unreliable on small data.

Comparison of estimation methods (same classifier)

Method	Estimated accuracy	Comment
Holdout (5 splits)	87.1% (range 85.0%–89.0%)	High variance; wastes 30% of data
10-fold cross-validation	85.5%	Uses all data; low bias; less variance
.632 bootstrap (50 resamples)	89.5% (SD 1.1)	Optimistic for an unpruned-style tree, because 36.8% of the weight comes from resubstitution accuracy (≈ 100%)
Majority-class baseline	88.0%	Reference: no model at all

Notes: the cross-validation estimate (85.5%) is below the 88% majority-class baseline, showing that accuracy alone makes a poor tree look acceptable only when compared with nothing. The bootstrap's higher value arises from its resubstitution component, so it should be trusted less for low-bias, high-variance learners such as unpruned trees.

Paired comparison of J48, Naïve Bayes and SMO (repeated 10-fold CV × 5, corrected resampled t-test, α = 0.05)

Classifier	Mean accuracy	SD
J48-like tree	86.0%	3.0%
Naive Bayes	90.1%	1.7%
SVM (RBF, scaled)	90.1%	1.2%

Comparison	Difference	t	p-value	Significant?
J48-like tree vs Naive Bayes	-4.1 pts	-3.49	0.001	Yes

Comparison	Difference	t	p-value	Significant?
J48-like tree vs SVM (RBF, scaled)	-4.1 pts	-3.55	0.001	Yes
Naive Bayes vs SVM (RBF, scaled)	0.0 pts	0.08	0.937	No

Naïve Bayes and SVM are significantly better than the tree (≈ +4 points, p ≈ 0.001), but **not different from each other** (p = 0.94). Both are only ≈ 1.4 points above the 88% baseline, so recall/AUC must also be examined before concluding which is useful. (In Weka: Experimenter → New → add dataset and the three classifiers → 10 runs × 10-fold CV → Analyse, test base = J48; the 'v' and '∗' marks in the output flag significant differences.)

ROC curve

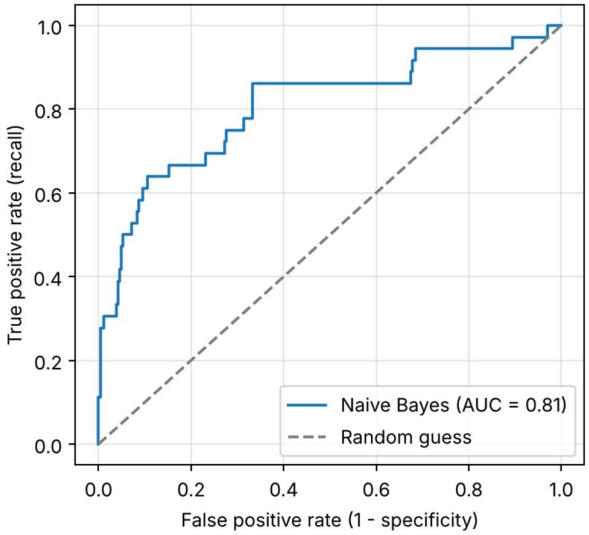


Figure 6.1 – ROC curve of Naïve Bayes on a 30% test set (AUC = 0.81).

The curve lies above the diagonal, so the classifier ranks fraud above non-fraud better than chance (AUC 0.81) even though its accuracy is hardly above the baseline.

Individual Verification

- **Why accuracy alone misleads:** with rare positives, a useless classifier scores ≥ 94% (here: 94% vs 95%). Report recall, precision, F-measure, AUC or cost.
- **Which estimation method to trust:** stratified 10-fold cross-validation (repeated if possible): it uses all the data, has low bias and moderate variance; holdout is too variable (4-point range above); .632 bootstrap is useful for very small samples but optimistic with overfitting learners.

Q7 – Day-4 Design Challenge: Design a Classification Solution (10 Marks)

QUESTION

A hospital wants to identify patients likely to be readmitted within 30 days. Only 6% are readmitted. A false negative is ten times more costly than a false positive. Doctors must understand the reason for each alert. Data: age, diagnosis group, length of stay, prior admissions, lab results, discharge medications (some values missing).

Stage 1: each student independently proposes technique, preprocessing (imbalance, missing values), evaluation measure/estimation method, explanation method. **Stage 2:** compare the designs. **Stage 3:** joint final design. **Stage 4:** A – analytical validation (expected cost of two thresholds, why accuracy is inappropriate, strengths/limits); B – experimental validation (design vs baseline, CV, recall/precision/AUC/cost, imbalance-handling effect). **Stage 5:** cross-defence. **Stage 6:** joint evaluation and conclusion.

ANSWER

NOTE

Stage 1 must contain each student's **own independent** initial proposal. The two proposals below are drafts written so the document is complete – each student should replace or edit their proposal with what they actually decided before discussing with the partner (the assignment does not deduct marks for an imperfect initial design).

Stage 1 – Independent Design Proposals

Student A's initial proposal

- **Technique:** a pruned decision tree (J48/CART) – doctors can read the path that produced an alert.
- **Preprocessing:** missing values – tree handles them (surrogate/fractional instances) or fill with the median/mode; encode diagnosis group as categories; imbalance – oversample readmitted patients (SMOTE) until classes are balanced.
- **Evaluation:** recall and F-measure, estimated with 10-fold cross-validation.
- **Explanation:** show the tree path/rule that fired for each patient.

Student B's initial proposal

- **Technique:** logistic regression (or a Naïve Bayes) giving a risk probability per patient; compare with a random forest as an accuracy benchmark.
- **Preprocessing:** impute labs with the median and add a 'missing' indicator; standardise numeric attributes; use class weights 10:1 (cost-sensitive learning) instead of changing the data.
- **Evaluation:** AUC and total cost ($10 \cdot FN + FP$) using stratified 10-fold CV; choose the threshold that minimises cost.
- **Explanation:** display the odds ratios and the top contributing factors (age, prior admissions, length of stay) for each patient.

Stage 2 – Comparison of the Two Designs

Aspect	Agreement / difference	Strengths and weaknesses	Decision
Model	Both want interpretability; A = tree, B = logistic regression	Tree: if–then explanation, but unstable and may overfit; LR: stable, calibrated probabilities, linear effects only	Retain LR as primary model; use a shallow tree as a benchmark/rule summary
Imbalance	A oversamples (SMOTE); B uses cost weights	Oversampling duplicates/synthesises patients and distorts probabilities; weights use the actual cost ratio directly	Retain cost-sensitive weights; compare oversampling in experiments
Missing values	Both impute; B adds indicators	Missingness itself may carry information (test not ordered = patient looked healthy)	Retain median imputation + indicator
Evaluation	A: recall/F1; B: AUC + cost	Recall alone ignores false alarms; F1 ignores the 10:1 ratio	Retain total cost + recall, precision, AUC
Missing cases	Neither discussed calibration, threshold or drift	Probabilities must be calibrated to use a cost threshold; model needs monitoring	Add calibration check and threshold derivation

Stage 3 – Joint Final Design

- **Technique:** L2-regularised **logistic regression** (interpretable risk score), with a shallow decision tree (depth 3–4) as a rule-based companion for documentation; random forest only as an accuracy benchmark.
- **Preprocessing:** median/mode imputation with missing-value indicator columns; one-hot encoding of diagnosis group and discharge medication; z-score standardisation of numeric attributes; remove identifiers.
- **Imbalance and cost:** cost-sensitive learning (class weight 10 for readmitted) **or** an unweighted model with a cost-based threshold. Using misclassification costs C_FN = 10 and C_FP = 1, flag a patient when P(readmit) > C_FP/(C_FP + C_FN) = 1/11 ≈ **0.09**.
- **Parameters:** regularisation strength C chosen by nested cross-validation; threshold chosen on validation data (≈ 0.09–0.12); calibration check (reliability curve).
- **Evaluation:** stratified 10-fold cross-validation (repeated), measures: recall, precision, ROC/AUC and **total cost = 10·FN + FP**; compare against a 'flag nobody' baseline and a default-threshold baseline.
- **Explanation to doctors:** for each alert show the risk probability, the top 3 contributing factors (coefficient × value), and the nearest tree rule, e.g. '3 prior admissions and length of stay > 7 days'.

Stage 4A – Student A: Analytical Validation

Expected cost of two thresholds on an illustrative cohort of 1,000 patients (6% = 60 readmitted, 940 not). The confusion matrices are assumed values typical of a model with a modest AUC; cost = 10·FN + 1·FP.

Decision rule	TP	FN	FP	TN	Accuracy	Recall	Precision	Total cost
Threshold 0.50	12	48	8	932	94.4%	20.0%	60.0%	10×48 + 8 = **488**
Threshold 0.10	42	18	170	770	81.2%	70.0%	19.8%	10×18 + 170 = **350**
Threshold 0.05 (extra)	52	8	330	610	66.2%	86.7%	13.6%	10×8 + 330 = **410**
Flag nobody	0	60	0	940	94.0%	0%	–	600

Decision rule	TP	FN	FP	TN	Accuracy	Recall	Precision	Total cost
Flag everybody	60	0	940	0	6.0%	100%	6.0%	940

The lower threshold (0.10) has the **lowest cost (350)**, 28% less than the default 0.50 (488) and 42% less than flagging nobody (600), even though its accuracy is 13 points lower. Lowering the threshold further (0.05) raises the cost again, because the extra false positives outweigh the few extra detected readmissions.

- **Why accuracy is inappropriate:** 'flag nobody' achieves 94% accuracy and would be selected over every useful model; accuracy ignores the 10:1 cost ratio and the minority class (6%). Accuracy ranks the 0.50 threshold (94.4%) first although it is the costliest working option.
- **Strengths of logistic regression here:** interpretable coefficients (odds ratios), calibrated probabilities, which allow a cost-based threshold; low variance, stable on 6% positives; easy missing-indicator handling.
- **Limitations:** assumes additive linear effects on the log-odds (interactions such as diagnosis × age must be added manually); sensitive to outliers/scaling; probabilities can be poorly calibrated when class weights are used; performance below a non-linear ensemble if strong interactions exist; correlated labs make coefficients unstable.

Stage 4B – Student B: Experimental Validation

Data: a public analogue could not be downloaded in the preparation environment, so a synthetic readmission-like data set was generated: 6,000 patients, 12 attributes (6 informative), 7.4% positives after label noise, 15% missing values in four lab-like attributes. Student B should repeat with the UCI Diabetes 130-US hospitals data in Weka/Python and compare. Evaluation: out-of-fold predictions from stratified 10-fold CV. Cost = 10·FN + FP. All models use median imputation + missing indicators.

Model	Thr.	TP	FN	FP	Recall	Precision	AUC	Total cost
Baseline: decision tree, default 0.5	0.50	84	361	136	18.9%	38.2%	0.709	3746
Baseline: logistic regression, default 0.5	0.50	1	444	5	0.2%	16.7%	0.757	4445
Final: LR + threshold 1/11 (0.09)	0.09	288	157	1363	64.7%	17.4%	0.757	2933
Final: LR + 10:1 class weights, 0.5	0.50	294	151	1373	66.1%	17.6%	0.760	2883
Final: LR + random oversampling, 0.5	0.50	332	113	1783	74.6%	15.7%	0.759	2913

Reference costs: flag nobody = 4450; flag everybody = 5555.

- **Baselines fail:** the default logistic regression flags almost nobody (recall 0.2%, cost 4,445 ≈ flag nobody 4,450); the default decision tree catches 19% of readmissions (cost 3,746).
- **Imbalance handling works:** the three final variants reach recall 65–75% at precision 16–18% and reduce cost to 2,883–2,933, i.e. **34–35% lower** than 'flag nobody' and 22–35% lower than the baselines. AUC changes little (0.757–0.760) – the gain comes from the decision rule, not from a better ranking.
- **Cost-based threshold 1/11 = 0.09** on the unweighted model gives about the same cost (2,933) as 10:1 class weights (2,883) and oversampling (2,913), confirming the theoretical threshold. The oversampling model has the highest recall (74.6%) but also the most false alarms (1,783).

Threshold sweep (unweighted logistic regression, out-of-fold):

Threshold	TP	FN	FP	Total cost
0.030	416	29	3910	4200
0.050	383	62	2746	3366
0.091	288	157	1363	2933
0.120	215	230	856	3156
0.150	148	297	572	3542
0.200	63	382	295	4115
0.300	14	431	91	4401
0.500	1	444	5	4445

The cost is minimised around 0.09 (2,933), as predicted by the 1/11 rule, rising on both sides (0.05: 3,366; 0.12: 3,156; 0.50: 4,445).

Stage 5 – Cross-Defence

Student A defends – what experimental evidence supports the design? Out-of-fold results show that (i) default-threshold models are useless for this task (cost ≈ 4,445 ≈ doing nothing), (ii) the cost-sensitive design lowers cost by about a third (4,450 → 2,933) while catching 65% of readmissions, (iii) the empirical cost minimum lies at the theoretically derived threshold, and (iv) all three imbalance methods give similar cost, so the choice can be made on interpretability.

Student B defends – how were the cost ratio and threshold derived from the confusion matrix? The costs enter as C_FN = 10 and C_FP = 1 (C_TP = C_TN = 0). For a patient with readmission probability p: expected cost of not flagging = p · 10 (risk of an FN), expected cost of flagging = (1 − p) · 1 (risk of an FP). Flag when 10p > 1 − p, i.e. p > 1/11 = 0.091. For a whole data set the confusion matrix gives the total cost 10·FN + FP at each threshold; the minimum of this curve (threshold sweep above and 4A) confirms the derived value. The threshold depends on the cost ratio, not on the prevalence, provided probabilities are calibrated.

Stage 6 – Joint Evaluation

- **Why standard accuracy-driven classifiers struggle:** with 6% positives, maximising accuracy leads to 'never flag'; the learner minimises 0/1 loss, which treats an FN and an FP equally, contrary to the 10:1 costs; few positive examples cause high variance; missing labs and noisy labels limit separability.
- **What we sacrificed for interpretable alerts:** potentially higher accuracy/AUC of a black-box ensemble or neural network that captures interactions; precision (only ≈ 17% of alerts are true readmissions – 5 of 6 alerts are false alarms, accepted deliberately because FN costs 10×); some probability calibration.
- **When the design would fail:** (i) the readmission rate or the cost ratio changes (e.g. prevalence falls to 0.5% → precision drops sharply and the alert load becomes unmanageable; the threshold must be re-derived), (ii) data drift – new coding, new hospital or medications, (iii) strong non-linear interactions not in the model, (iv) missingness patterns change, (v) labels depend on treatment given after the alert (feedback loop), (vi) alert fatigue if false alarms exceed clinical capacity.

Conclusion – what to examine before choosing a classifier:

- **Data characteristics:** size, number and type of attributes, missing values, noise, class imbalance, correlation/interactions, linear vs non-linear separability, need for scaling.

- **Business requirements:** misclassification costs, required explainability/regulation, acceptable alert volume, need for probabilities or rankings, training/prediction speed, need to update the model, and the right evaluation measure (cost, recall/precision, AUC – not accuracy).

Contribution Record

The table below reflects the planned division described in the assignment. **Edit it so that it shows the actual work done**; 'primary contribution' does not mean exclusive responsibility.

Question	Student A – Primary Contribution	Student B – Primary Contribution	Joint Work
Q1	Manual gains, Gini/gain ratio, tree and paths, ID attribute	J48 on 12 tuples and credit-g, pruning experiments, overfitting	Attributes, criterion, pruning approach, Weka set-up
Q2	Priors, conditional probabilities, posteriors, Laplace correction	Weka NaiveBayes check, correlated-attribute experiment	Naïve Bayes vs tree conclusion
Q3	Rule coverage/accuracy, conflict resolution, sequential covering	JRip/PART vs J48, class-association rules, support/confidence	Tree-to-rules; recommendation for loan officer
Q4	Hand calculation of one iteration, weight initialisation	MLP experiments, overfitting, tree comparison	Network set-up; back-propagation explanation
Q5	Support vectors, hyperplane, margin, C and kernels	Kernel/C/γ experiments, scaling	Plot; SVM conclusion
Q6	Metrics, MAE/MSE/RMSE, class imbalance, ROC explanation	Holdout/CV/bootstrap, paired comparison, ROC plot	Discussion of meaningful measure
Q7	Analytical validation (expected cost, threshold), defence of experiments	Experimental validation, defence of threshold derivation	Stages 1–3 and 6, comparison record